Description Submission Düzenle Submission view

Available from: Salı, 21 Nisan 2026, 3:00 PM
Due date: Pazar, 3 Mayıs 2026, 11:59 PM
Requested files: Assignment2.java (Download)
Type of work: Individual work

CSE102T Assignment 2

AIRPORT MANAGEMENT SYSTEM

Please read this manual carefully before beginning your implementation in the Moodle VPL environment.

Overview and Learning Objectives

In this assignment, you will implement a system that utilizes **Polymorphism**, **Interfaces**, **Bounded Generics**, and various **Data Structures** (such as Stacks, Queues, Priority Queues, Sets, and Maps). You will build subsystems for baggage handling, flight logistics, and air traffic control.

Deliverables

Submit a single Java file named Assignment2.java via LMS System (Moodle) by the due
date and time specified. Late submissions will be subject to the course's late policy. Submissions via MS Teams will not be graded.
https://lms.akdeniz.edu.tr

Section 1: Interfaces

The system relies on two core interfaces to establish common behaviors.

- **Prioritizable Interface**
  - **Purpose:** Defines a contract for objects that require a dynamically calculated priority level.
  - **Methods:** Contains a single method named calculatePriority that takes no parameters and returns a double representing the priority score.
- **SecurityClearance Interface**
  - **Purpose:** Defines a contract for entities that must pass a security screening.
  - **Methods:** Contains a single method named performSecurityCheck that takes no parameters and returns a boolean.

Section 2: Passenger and Baggage Subsystem

Baggage Class

Represents a single piece of luggage.

- **Attributes:** A string for the tag ID, a double for the weight, and a boolean flag (isCleared) to track security status.
- **Constructor:** Initializes the tag ID and weight. **Crucially, the clearance flag must initially be set to false.**

?

Methods:

- Standard getters for tag ID and weight.
- **Security Check:** Evaluates the weight. Bags weighing **exactly 100.0 or less** pass the check. Bags over 100.0 (e.g., 100.1 or extreme weights like 5000.0) fail. If it passes, the clearance flag becomes true and the method returns true. If it fails, the flag remains false and the method returns false.

Passenger Class

Represents a traveler.

- **Constants:** Must include static integer constants: FIRST_CLASS (3), BUSINESS (2), and ECONOMY (1).
- **Attributes:** A string for the passport ID, a string for the name, an integer for the ticket priority, and a list of Baggage objects.
- **Constructor:** Initializes passport ID, name, and ticket priority. The baggage list must be initialized as an empty list.
- **Methods:**
  - addBaggage: Takes a Baggage object and adds it to the list.
  - Standard getters for bags, name, passport ID, and ticket priority.
  - **Security Check:** Iterates through all bags. The passenger passes only if *all* bags pass. **Note:** A passenger with an empty baggage list automatically passes security.
  - **Comparison:** Sorts passengers in descending order based on ticket priority (First Class comes before Business, which comes before Economy). Passengers with equal priority should return 0.

CargoPayload Class

Represents freight items.

- **Attributes:** A string for the manifest ID, a double for the tonnage, and a boolean for whether it is hazardous.
- **Constructor:** Initializes all three attributes.
- **Methods:** Standard getters for the manifest ID, tonnage, and hazardous status.
- **Note:** The system must accept 0.0 or even negative values for tonnage without throwing errors.

Section 3: Flight Subsystem

Flight Class (Abstract)

The foundational class for all aircraft.

- **Constants:** Static string constants for status: Scheduled, Boarding, In Flight, Landed, and Docked.
-

Attributes: String flight ID, string destination, double remaining fuel, and string status.

- **Constructor:** Initializes ID, destination, and fuel. Initial status must be Scheduled.
- **Methods:**
  - Standard getters for ID, destination, and status.
  - Setter for status.
  - An abstract method prepareForDeparture returning nothing.
  - **Comparison:** Natural ordering is alphabetical by destination. If destinations are identical, order alphabetically by flight ID.

CommercialFlight Class

- **Attributes:** A priority queue for boarding passengers, a stack for the cargo hold, and a list for seated passengers.
- **Constructor:** Initializes inherited fields and empty data structures.
- **Methods:**
  - queueForBoarding: Takes a Passenger. Added to the queue *only* if they pass security.
  - prepareForDeparture: Changes status to Boarding. Removes passengers from the queue, adds them to the seated list, and pushes their bags into the cargo hold stack (Last-In-First-Out). Resets status to Scheduled when done.
  - getCargoHold: Returns the baggage stack.
  - calculatePriority: The formula is exactly: 10.0 + (100.0 / (remainingFuel + 1)). (e.g., 99.0 fuel yields a priority of 11.0; 0.0 fuel yields 110.0).

CargoFlight Class

- **Attributes:** A list of CargoPayload objects.
- **Methods:**
  - loadFreight: Takes a CargoPayload and adds it to the list.
  - prepareForDeparture: Sets status to Scheduled.
  - calculatePriority: Base priority is 5.0. Add 50.0 for *each* hazardous payload on board. Then add the fuel factor: (100.0 / (remainingFuel + 1)).

EmergencyFlight Class

- **Attributes:** A string for the emergency protocol.
- **Methods:**
  - prepareForDeparture: Sets status to Scheduled.
  - calculatePriority: Must return exactly 9999.9 to ensure absolute priority.

Section 4: Terminal Logistics

TerminalGate Class (Generic)

A bounded generic class (T extends Flight).

- **Attributes:** A string gate ID and a reference to the docked flight (T).
- **Methods:**
  - dockFlight: Takes a flight T. If the gate is empty, assigns the flight, changes its status to Docked, and returns true. If occupied, returns false. A gate can be re-docked after being released.
  - releaseFlight: Clears the docked flight and returns it. If the gate is already empty, it must safely return null.

Section 5: Air Traffic Control (ATC)

Manages the flow of the airport.

- **Attributes:**
  - An approach priority queue for landing.
  - A departure runway FIFO queue for takeoff.
  - A set of cleared passport IDs.
  - A map of destinations to sorted sets of flights.
  - A stack for the system audit log.
- **Methods:**
  - registerFlight: Adds the flight to the destination map and logs it.
  - requestLanding: Changes the flight's status to In Flight and adds it to the approach priority queue.
  - processNextLanding: Removes the highest priority flight, changes its status to Landed. **Must not throw an exception if the queue is empty.**

- requestTakeoff: Adds the flight to the departure FIFO queue (does *not* change status yet).
- processNextTakeoff: Removes the next flight from the runway and changes its status to In Flight. **Must not throw an exception if the queue is empty.**
- clearPassengerSecurity: Takes a Passenger. Only if they pass security is their passport ID added to the cleared set.

Section 6: Edge Cases & Boundary Conditions Summary

To ensure your code passes the automated grader, verify the following strict boundaries:

1. **Baggage Limits:** Exactly 100.0 is safe. 100.1 is a failure.
2. **Initial States:** A newly created Baggage must have an isCleared state of false before performSecurityCheck is called.
3. **Empty Baggage Lists:** A passenger carrying 0 bags inherently poses no security risk and must pass their security check.
4. **Fuel Depletion:** A flight with 0.0 fuel must not cause a division-by-zero error in the priority calculation (ensure you are using fuel + 1).
5. **Cargo Anomalies:** CargoPayload tonnage can safely accept 0.0 or negative inputs representing logistical adjustments. Do not restrict this.
6. **Empty ATC Queues:** Calling processNextLanding or processNextTakeoff when no flights are queued must fail gracefully (do nothing) rather than crashing the system.
7. **Stack LIFO Ordering:** When pushing a passenger's bags to the commercial flight's cargo hold, if bag A is added to the passenger then bag B, popping from the cargo hold must yield bag B first, then bag A.

Section 7: Final Submission Checklist

Before submitting your code to the autograder, please use this checklist to verify that your implementation meets all the strict requirements and edge cases of the system.

1. Initialization & State Management

- [ ] **Baggage State:** Is the isCleared flag explicitly initialized to false in the Baggage constructor?
- [ ] **Collections:** Are all lists, queues, stacks, sets, and maps properly initialized in their respective constructors to prevent NullPointerExceptions?
- [ ] **Default Status:** Does the Flight constructor set the initial status to Scheduled?

2. Security & Logic Boundaries

- [ ] **Weight Limits:** Does performSecurityCheck in Baggage return true for exactly 100.0 and false for anything strictly greater than 100.0?
- [ ] **Empty Bags:** Does a Passenger with an empty baggage list successfully pass their security check?
- [ ] **Strict Failure:** If even *one* bag fails security, does the Passenger immediately fail their overall security check?

3. Priority Calculations

- [ ] **Division by Zero:** Did you use (remainingFuel + 1) in your division formulas to prevent crash errors when fuel is 0.0?
- [ ] **Hazmat Accumulation:** In CargoFlight, is the 50.0 hazardous material bonus applied for *every single* hazardous payload in the freight list?
- [ ] **Emergency Value:** Does EmergencyFlight.calculatePriority() return exactly 9999.9?

4. Data Structures & Ordering

- [ ] **LIFO Cargo:** Are you using a Stack for the CommercialFlight cargo hold? When pushing a passenger's bags, are you ensuring they are popped in the correct Last-In-First-Out order?
- [ ] **Passenger Sorting:** Are passengers ordered in *descending* priority (First Class boards before Economy)?
- [ ] **Flight Sorting:** Are flights ordered alphabetically by destination first, and then alphabetically by flight ID if the destinations match?
- [ ] **ATC Queues:** Are you using a PriorityQueue for incoming landings and a standard FIFO Queue for the departure runway?

5. System Robustness (Edge Cases)

- [ ] **Empty ATC Queues:** Do processNextLanding() and processNextTakeoff() simply do nothing (return safely) if their respective queues are empty?
- [ ] **Empty Gate Releases:** Does TerminalGate.releaseFlight() safely return null if the gate is already empty?
- [ ] **Negative Tonnage:** Does CargoPayload accept 0.0 or negative numbers for tonnage without throwing an error?
- [ ] **Gate Occupancy:** Does dockFlight correctly reject (return false) a flight if a different flight is already docked at that gate?

If you can confidently check all these boxes, your system is well-prepared for the autograder. Good luck with your final testing!

Section 8: Example Scenarios for Self-Verification

Scenario A: Baggage Edge Case & Security
A First Class passenger arrives with two bags: Bag 1 weighs 99.9, and Bag 2 weighs 100.0.
Result: Both bags fall within the acceptable limit. The passenger passes security. If Bag 2 weighed 100.1, the passenger would fail, and queueForBoarding would ignore them.
Scenario B: Complex ATC Priority Resolution
ATC receives landing requests from three flights:

1. Commercial Flight with 99.0 fuel (Priority calculates to exactly 11.0).
2. Cargo Flight with 4.0 fuel and one Hazardous payload (Priority calculates to 5.0 base + 50.0 hazmat + 20.0 fuel factor = 75.0).
3. Emergency Flight (Priority calculates to 9999.9).
   Result: Upon calling processNextLanding three consecutive times, the Emergency flight lands first, followed by the Cargo flight, and finally the Commercial flight.

Scenario C: Gate Management
Terminal Gate G1 docks Flight TK1. A second flight, TK2, attempts to dock at G1.

Result: The dockFlight method for TK2 returns false. Gate G1 must call releaseFlight (which returns TK1) before TK2 can successfully dock. If releaseFlight is called twice in a row, the second call safely returns null.

Example Output:

```
=== GLOBAL AIRPORT MANAGEMENT SYSTEM INITIALIZED ===

--- TESTING PASSENGER SECURITY & BOARDING ---
ALERT: Bag BAG-C1 is too heavy and failed security.

Flight TK1902 docked at Gate A1
Ahmet Yilmaz queued for boarding (Priority: 1).
John Doe queued for boarding (Priority: 3).
ALERT: Bag BAG-C1 is too heavy and failed security.
Jane Smith FAILED security clearance. Cannot board.

--- Boarding TK1902 ---
Boarded: John Doe | First Class
Boarded: Ahmet Yilmaz | Economy

--- TESTING CARGO LOGISTICS ---
Flight FX11 docked at Gate C1
Loaded cargo MFST-001 onto FX11
Loaded cargo MFST-HAZ-002 onto FX11
--- Securing Cargo for FX11 ---
HAZMAT PROTOCOL ACTIVE for payload: MFST-HAZ-002

--- TESTING AIR TRAFFIC CONTROL ---
Flight TK1902 has left Gate A1
Flight FX11 has left Gate C1

ATC CLEARANCE: TK1902 is cleared for takeoff to Istanbul

ATC CLEARANCE: FX11 is cleared for takeoff to Memphis

>>> INITIATING LANDING SEQUENCE <<<

ATC CLEARANCE: LIFE-99 is cleared to land. (Fuel: 40.0%)

ATC CLEARANCE: FX11 is cleared to land. (Fuel: 20.0%)

ATC CLEARANCE: TK1902 is cleared to land. (Fuel: 85.0%)

=== DAILY AUDIT LOG (Most Recent First) ===
 - Flight TK1902 landed successfully.
 - Flight FX11 landed successfully.
 - Flight LIFE-99 landed successfully.
 - Flight LIFE-99 requested landing.
 - Flight FX11 requested landing.
 - Flight TK1902 requested landing.
 - Flight FX11 took off.
 - Flight TK1902 took off.
 - Flight FX11 entered departure runway.
 - Flight TK1902 entered departure runway.
 - Security cleared passenger: US-882
 - Security cleared passenger: TR-991
 - Registered flight: LIFE-99
 - Registered flight: FX11
 - Registered flight: TK1902
```

Good luck with your implementation!

Requested files

Assignment2.java

```java
import java.util.*;

public class Assignment2 {
    public static void main(String[] args) {
        System.out.println("=== GLOBAL AIRPORT MANAGEMENT SYSTEM INITIALIZED ===\n");

        AirTrafficControl atc = new AirTrafficControl();

        // 1. Create Gates
        TerminalGate<CommercialFlight> gateA1 = new TerminalGate<>("A1");
        TerminalGate<CargoFlight> gateC1 = new TerminalGate<>("C1");

        // 2. Create Flights
        CommercialFlight commFlight = new CommercialFlight("TK1902", "Istanbul", 85.0);
        CargoFlight cargoFlight = new CargoFlight("FX11", "Memphis", 20.0); // Low fuel!
        EmergencyFlight medevac = new EmergencyFlight("LIFE-99", "Ankara", 40.0, "Organ Transport");

        atc.registerFlight(commFlight);
        atc.registerFlight(cargoFlight);
        atc.registerFlight(medevac);

        // 3. Passenger & Baggage Subsystem Test
        System.out.println("--- TESTING PASSENGER SECURITY & BOARDING ---");
        Passenger pass1 = new Passenger("TR-991", "Ahmet Yilmaz", Passenger.ECONOMY);
        pass1.addBaggage(new Baggage("BAG-A1", 20.5)); // Safe

        Passenger pass2 = new Passenger("US-882", "John Doe", Passenger.FIRST_CLASS);
        pass2.addBaggage(new Baggage("BAG-B1", 15.0)); // Safe

        Passenger pass3 = new Passenger("UK-773", "Jane Smith", Passenger.BUSINESS);
        pass3.addBaggage(new Baggage("BAG-C1", 105.0)); // Too Heavy! Fails security.

        // Process Security
        atc.clearPassengerSecurity(pass1);
        atc.clearPassengerSecurity(pass2);
        atc.clearPassengerSecurity(pass3); // Fails

        System.out.println();
        // Dock and Board
        gateA1.dockFlight(commFlight);
        commFlight.queueForBoarding(pass1);
        commFlight.queueForBoarding(pass2);
        commFlight.queueForBoarding(pass3); // Rejected at gate

        System.out.println();
        // Execute Boarding - First class boards before Economy!
        commFlight.prepareForDeparture();

        // 4. Cargo Logistics Test
        System.out.println("\n--- TESTING CARGO LOGISTICS ---");
        gateC1.dockFlight(cargoFlight);
        cargoFlight.loadFreight(new CargoPayload("MFST-001", 50.0, false));
        cargoFlight.loadFreight(new CargoPayload("MFST-HAZ-002", 10.0, true)); // Hazmat
        cargoFlight.prepareForDeparture();

        // 5. Air Traffic Control Test (Priority Queues)
        System.out.println("\n--- TESTING AIR TRAFFIC CONTROL ---");

        // Flights releasing from gates and requesting takeoff
        atc.requestTakeoff(gateA1.releaseFlight());
        atc.requestTakeoff(gateC1.releaseFlight());

        // Takeoffs process sequentially (FIFO Queue)
        atc.processNextTakeoff();
        atc.processNextTakeoff();

        // Incoming Flights request landing (PriorityQueue)
        atc.requestLanding(commFlight);
        atc.requestLanding(cargoFlight);
        atc.requestLanding(medevac); // Arrives late but jumps the queue!

        // Landing Order Expected: Medevac (9999), Cargo (Hazmat + Low Fuel), CommFlight (Safe Fuel)
        System.out.println("\n>>> INITIATING LANDING SEQUENCE <<<");
        atc.processNextLanding();
        atc.processNextLanding();
        atc.processNextLanding();

        // 6. System Reporting (Stack)
        atc.printDailyAuditLog();
    }
}

//CODE HERE
```

VPL